Language Models

Nagesh

May 2026

"WISDOM WITHOUT COMPASSION IS BARREN
COMPASSION WITHOUT WISDOM IS BLINDNESS"
(A Buddist teaching)

Contents

Introduction

Many real-world datasets are sequential in nature.

Examples include:

- Natural language sentences
- Speech signals
- Time series
- DNA / protein sequences
- Video frames

A sequence can be represented as

$$X = \{x_1, x_2, x_3, \ldots, x_T\}$$

where:

- T is the sequence length
- x_t is the observation at time step t

Each sequence X is sampled from an unknown data distribution:

$$X \sim p_x$$

Goal of generative modeling:

1. Learn the underlying distribution p_x
2. Generate new realistic samples from it

Since the true distribution p_x is unknown and intractable, we approximate it using a parameterized model:

$$p_\theta(X)$$

where θ denotes learnable parameters.

The learning objective is:

$$\theta^* = \arg\min_\theta D_f(p_x \parallel p_\theta)$$

where:

- D_f is a divergence measure
- Examples: KL divergence, Jensen-Shannon divergence

Using maximum likelihood estimation (MLE),

$$\theta^* = \arg\max_\theta \mathbb{E}_{X\sim p_x} [\log p_\theta(X)]$$

Equivalently,

$$\theta^* = \arg\min_\theta -\mathbb{E}_{X\sim p_x} [\log p_\theta(X)]$$

Thus, learning reduces to maximizing the probability assigned to observed data.

Autoregressive Models

Autoregressive (AR) models factorize a joint distribution into a product of conditional distributions.

Using the chain rule of probability,

$$p_\theta(X) = p_\theta(x_1, x_2, \ldots, x_T)$$

can be written as

$$p_\theta(X) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t})$$

where

$$x_{<t} = \{x_1, x_2, \ldots, x_{t-1}\}$$

This means:

- Predict the next token/value
- Conditioned on all previous observations

Generation process:

$$x_1 \to x_2 \to x_3 \to \cdots \to x_T$$

Hence autoregressive models generate sequences sequentially.

Classical Autoregressive (AR) Model

A classical AR(p) process assumes:

$$x_t = \sum_{i=1}^{p} a_i x_{t-i} + \epsilon_t$$

where:

- p = order of the autoregressive model
- a_i = learnable coefficients
- ϵ_t = Gaussian noise

Noise model:

$$\epsilon_t \sim \mathcal{N}(0, \sigma^2)$$

Interpretation:

- Current value depends linearly on previous values
- Noise introduces stochasticity

Example:
For AR(2),

$$x_t = a_1 x_{t-1} + a_2 x_{t-2} + \epsilon_t$$

Limitations:

- Linear dynamics only
- Cannot model complex long-range dependencies

Neural Autoregressive Models

Instead of linear dependence, we use neural networks to model conditionals.

$$p_\theta(x_t \mid x_{<t}) = \text{NeuralNetwork}_\theta(x_{<t})$$

The neural network outputs:

- Probability distribution
- Mean and variance
- Next token logits

Possible architectures:

- Multi-Layer Perceptrons (MLP)
- Convolutional Neural Networks (CNN)
- Transformers

Advantages:

- Nonlinear modeling
- High expressive power
- Can model complex data distributions

Recurrent Neural Networks (RNN)

RNNs introduce a hidden state to summarize past information.

Hidden state update:

$$h_t = f_\theta(h_{t-1}, x_{t-1})$$

Output generation:

$$x_t = g_\phi(h_t)$$

where:

- h_t = hidden state
- f_θ = recurrent transition function
- g_ϕ = output function

Both functions are nonlinear and learnable.

Examples:

- Vanilla RNN
- LSTM
- GRU

Sequential Computation in RNN

Initial hidden state:

$$h_0 = 0$$

First step:

$$h_1 = f_\theta(h_0, x_0)$$

$$x_1 = g_\phi(h_1)$$

Second step:

$$h_2 = f_\theta(h_1, x_1)$$

$$x_2 = g_\phi(h_2)$$

General recurrence:

$$h_t = f_\theta(h_{t-1}, x_{t-1})$$

$$x_t = g_\phi(h_t)$$

Parameter Sharing in RNNs

A major advantage of RNNs is parameter sharing across time.

The same parameters:

$$\theta, \phi$$

are reused for all time steps.

Benefits:

- Fewer parameters
- Handles variable-length sequences
- Better generalization

This temporal parameter sharing is a key idea in sequence modeling.

Training Objective of Autoregressive Models

Using maximum likelihood estimation:

$$\mathcal{L}(\theta) = -\log p_\theta(X)$$

Using autoregressive factorization:

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

Thus the total loss is the sum of next-step prediction losses.

For discrete tokens:

$$\mathcal{L} = \text{CrossEntropyLoss}$$

For continuous signals:

$$\mathcal{L} = \text{Negative Log Likelihood}$$

Sampling from an Autoregressive Model

Generation proceeds sequentially.

Step 1:

$$x_1 \sim p_\theta(x_1)$$

Step 2:

$$x_2 \sim p_\theta(x_2 \mid x_1)$$

Step 3:

$$x_3 \sim p_\theta(x_3 \mid x_1, x_2)$$

Continue until:

$$x_T \sim p_\theta(x_T \mid x_{<T})$$

Hence sequence generation is causal and iterative.

Advantages of Autoregressive Models

- Exact likelihood computation
- Stable training
- Strong generative performance
- Natural sequence modeling

Limitations of Autoregressive Models

- Sequential generation is slow
- Cannot parallelize sampling
- Long-range dependencies are difficult
- Error accumulation during generation

These limitations motivated later architectures:

- Transformers
- Diffusion models
- Flow models
- State-space models

Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are designed for sequential data.
Examples include:

- Natural language
- Time series
- Speech

- Protein sequences

The key idea behind RNNs is:

Parameter sharing across time

Given a sequence

$$X = \{x_1, x_2, x_3, \ldots, x_T\}$$

the same recurrent cell is repeatedly applied at every time step.

Single RNN Cell

At time step t,
the hidden pre-activation is

$$z_t = W_h h_{t-1} + W_x x_t + b_1$$

where:

- x_t = input at time t
- h_{t-1} = previous hidden state
- W_h = recurrent weight matrix
- W_x = input projection matrix

The hidden state is obtained using a nonlinear activation:

$$h_t = \sigma(z_t)$$

where σ is typically:

- sigmoid
- tanh

The output prediction is

$$\hat{x}_t = W_o h_t + b_2$$

Thus the recurrent update is:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b_1)$$

Backpropagation Through Time (BPTT)

Training RNNs requires gradients through all previous time steps.

This process is called:

Backpropagation Through Time (BPTT)

Since hidden states recursively depend on previous hidden states, the computational graph becomes very deep in time.

Forward Dynamics

For simplicity consider:

$$z_t = W_h h_{t-1} + W_x x_t$$

$$h_t = \sigma(z_t)$$

Suppose the loss at the final time step is:

$$L_T$$

We want to compute:

$$\frac{\partial L_T}{\partial h_t}$$

for an earlier hidden state h_t.

Gradient Propagation Through Time

Using the chain rule:

$$\frac{\partial L_T}{\partial h_t} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_T}{\partial h_t}$$

Thus the critical quantity is:

$$\frac{\partial h_T}{\partial h_t}$$

Because hidden states recursively depend on each other,

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k}$$

This product of Jacobians determines how gradients propagate.

Single Step Jacobian

Now derive:

$$\frac{\partial h_{k+1}}{\partial h_k}$$

We have:

$$h_{k+1} = \sigma(z_{k+1})$$

with

$$z_{k+1} = W_h h_k + W_x x_{k+1}$$

Applying chain rule:

$$\frac{\partial h_{k+1}}{\partial h_k} = \frac{\partial h_{k+1}}{\partial z_{k+1}} \frac{\partial z_{k+1}}{\partial h_k}$$

Now,

$$\frac{\partial z_{k+1}}{\partial h_k} = W_h$$

and

$$\frac{\partial h_{k+1}}{\partial z_{k+1}} = \operatorname{diag}\left(\sigma'(z_{k+1})\right)$$

Therefore,

$$\boxed{\frac{\partial h_{k+1}}{\partial h_k} = \operatorname{diag}\left(\sigma'(z_{k+1})\right) W_h}$$

Full Gradient Expression

Substituting into the temporal product:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t}^{T-1} \operatorname{diag}\left(\sigma'(z_{k+1})\right) W_h$$

This is the core mathematical reason behind vanishing gradients.

Vanishing Gradient Problem

Take norms on both sides:

$$\left\| \frac{\partial h_T}{\partial h_t} \right\| \leq \prod_{k=t}^{T-1} \|\text{diag}(\sigma'(z_{k+1}))\| \cdot \|W_h\|$$

using the submultiplicative property of matrix norms:

$$\|AB\| \leq \|A\|\|B\|$$

Bounding the Activation Derivative

For sigmoid activation:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Its derivative is:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Maximum value:

$$\sigma'(x) \leq \frac{1}{4}$$

Hence,

$$\|\text{diag}(\sigma'(z_{k+1}))\| \leq 1$$

(usually much smaller than 1).

Spectral Norm of the Recurrent Matrix

Let

$$\lambda_{\max}$$

be the largest singular value of W_h.
Then

$$\|W_h\| = \lambda_{\max}$$

Thus,

$$\left\| \frac{\partial h_T}{\partial h_t} \right\| \leq \prod_{k=t}^{T-1} \lambda_{\max}$$

which gives

$$\boxed{\left\| \frac{\partial h_T}{\partial h_t} \right\| \leq (\lambda_{\max})^{T-t}}$$

Vanishing Gradients

If

$$\lambda_{\max} < 1$$

then

$$(\lambda_{\max})^{T-t} \to 0$$

exponentially fast as:

$$T - t \to \infty$$

Thus gradients flowing from distant time steps vanish.
This prevents learning long-range dependencies.

Exploding Gradients

If

$$\lambda_{\max} > 1$$

then

$$(\lambda_{\max})^{T-t} \to \infty$$

leading to exploding gradients.
Thus vanilla RNNs are unstable for long sequences.

How to Solve Vanishing Gradients?

The core objective is:

Preserve gradient flow across time

Mathematically we want:

$$\frac{\partial h_{k+1}}{\partial h_k} \approx I$$

instead of repeated multiplication by small matrices.

Additive State Updates

Vanishing gradients arise because hidden states are updated multiplicatively.

The solution used in LSTMs, GRUs, and Residual Networks is:

Additive information flow

General additive recurrent update:

$$h_t = \alpha_t \odot h_{t-1} + \beta_t \odot \tilde{h}_t$$

where:

- $\odot$ denotes element-wise multiplication
- α_t = memory gate
- β_t = update gate
- $\tilde{h}_t$ = candidate hidden state

Candidate state:

$$\tilde{h}_t = f(W_x x_t + W_h h_{t-1})$$

Special Cases

$$\beta_t = 1$$

gives Highway Networks.

$$\beta_t = 1 - \alpha_t$$

gives GRUs.

Learnable gates:

$$\alpha_t, \beta_t$$

lead to LSTMs.

Gradient Flow in Additive Networks

Now derive the gradient carefully.

We have:

$$h_t = \alpha_t \odot h_{t-1} + \beta_t \odot \tilde{h}_t$$

Differentiate w.r.t. h_{t-1}:

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(\alpha_t) + \frac{\partial \alpha_t}{\partial h_{t-1}}\text{diag}(h_{t-1}) + \frac{\partial(\beta_t \odot \tilde{h}_t)}{\partial h_{t-1}}$$

Define all complicated nonlinear terms as:

$$\epsilon_t$$

Thus:

$$\boxed{\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(\alpha_t) + \epsilon_t}$$

Temporal Gradient Product

Now across time:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t}^{T-1} [\text{diag}(\alpha_{k+1}) + \epsilon_{k+1}]$$

If gates satisfy:

$$\alpha_t \approx 1$$

and nonlinear corrections are small,

$$\epsilon_t \approx 0$$

then

$$\frac{\partial h_t}{\partial h_{t-1}} \approx I$$

Thus:

$$\boxed{\frac{\partial h_T}{\partial h_t} \approx I}$$

and gradients can flow across very long sequences.

Residual Connection Interpretation

This idea is identical to residual networks.

Suppose:

$$y = x + f(x)$$

Differentiate:

$$\frac{dy}{dx} = 1 + \frac{df}{dx}$$

Even if:

$$\frac{df}{dx} \approx 0$$

we still have:

$$\frac{dy}{dx} \approx 1$$

Hence gradients never vanish completely.
This is the mathematical foundation behind:

- LSTMs
- GRUs
- Highway Networks
- Residual Networks (ResNets)
- Transformers with skip connections

Key Insight

Vanilla RNN:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t)$$

Gradient flow is multiplicative.

LSTM/GRU/Residual style updates:

$$h_t = h_{t-1} + f(h_{t-1}, x_t)$$

Gradient flow becomes additive.

This preserves long-range information and stabilizes training.

A Note on Transformation Learning

A central idea in machine learning is:

Learning useful transformations of data

This is also called:

- Representation learning
- Feature learning
- Embedding learning

What is Embedding Learning?

Suppose data lives in a d-dimensional space:

$$x \in \mathbb{R}^d$$

We apply a transformation:

$$\phi : \mathbb{R}^d \to \mathbb{R}^k$$

such that

$$z = \phi(x)$$

where:

- $z \in \mathbb{R}^k$
- z is called:
  - embedding
  - representation
  - latent feature

The goal is:

Represent data in a more meaningful space.

Representation Learning in Neural Networks

Consider a neural network:

$$h(x) = \sigma\left(W_2\sigma(W_1x + b_1) + b_2\right)$$

This can be interpreted as a sequence of learned transformations.

First Transformation

$$z_1 = W_1x$$

This is the first projection of the data.
The matrix W_1 defines a learned basis transformation.
After nonlinearity:

$$h_1 = \sigma(z_1)$$

This produces the first learned representation.

Second Transformation

Now project again:

$$z_2 = W_2 h_1$$

followed by another nonlinearity:

$$h_2 = \sigma(z_2)$$

This becomes a higher-level representation.
Thus every layer creates a new embedding of the data.

Deep Networks as Hierarchical Representation Learning

A deep neural network can therefore be viewed as:

$$x \to h_1 \to h_2 \to h_3 \to \cdots \to h_L$$

where:

$$h_l = \phi_l(h_{l-1})$$

Each layer learns:

- a new coordinate system
- a new feature space
- a new embedding

Thus deep learning is fundamentally:

Hierarchical embedding learning

Lower layers learn simple structures.
Higher layers learn abstract structures.

Example: Vision Models

For images:

$$x \to \text{pixels}$$

Early CNN layers learn:

- edges

- corners
- textures

Middle layers learn:

- object parts
- shapes

Deep layers learn:

- objects
- semantic concepts

Thus representations become increasingly abstract.

Example: Language Models

For text:

$$x = \text{word/token}$$

Initial embedding layer learns:

$$z = E[x]$$

where E is the embedding matrix.
Nearby embeddings encode semantic similarity:

$$\texttt{king} \approx \texttt{queen}$$

$$\texttt{cat} \approx \texttt{dog}$$

Transformer layers then progressively learn:

- syntax
- grammar
- semantics
- reasoning patterns

Fixed Transformations vs Learned Transformations

A major distinction in machine learning is:

Fixed basis transformations vs **Learned basis transformations**

Fixed Transformations

In classical methods:

$$\phi(x)$$

is predefined by the user.
The transformation does not adapt to data.
Examples include:

- Fourier Transform
- Wavelet Transform
- PCA
- Polynomial Kernels
- Radial Basis Functions
- Discrete Cosine Transform (DCT)
- Sine/Cosine Basis Expansions

Example: Fourier Transform

In Fourier analysis,
signals are projected onto fixed sinusoidal basis functions.

$$f(x) = \sum_{k=-\infty}^{\infty} c_k e^{ikx}$$

Basis functions:

$$e^{ikx}$$

are fixed beforehand.
The model does not learn the basis.
It only learns the coefficients:

$$c_k$$

Interpretation:

Project data onto predefined frequency directions.

Example: Principal Component Analysis (PCA)

PCA finds orthogonal directions of maximum variance.

Projection:

$$z = W^T x$$

where columns of W are principal components.
Although PCA learns projection directions from data, the transformation is:

- linear
- shallow
- non-adaptive after training

PCA cannot learn hierarchical nonlinear representations.

Example: Wavelet Transform

Wavelets project signals onto localized basis functions.

$$f(x) = \sum_{m,n} c_{m,n} \psi_{m,n}(x)$$

where:

$$\psi_{m,n}(x)$$

are shifted and scaled wavelets.
Applications:

- image compression
- denoising
- signal processing

Again the basis functions are predefined.

Example: Kernel Methods

In SVMs and kernel methods:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

The mapping:

$$\phi(x)$$

is fixed implicitly through the kernel.
Examples:

- Polynomial kernel
- Gaussian RBF kernel
- Laplacian kernel

The representation space is not learned end-to-end.

Neural Networks as Learnable Basis Transformations

Neural networks generalize all these ideas.
Instead of fixing basis functions manually,
the basis itself is learned from data.
Suppose:

$$z = Wx$$

Rows of W can be interpreted as learned basis vectors.
Unlike Fourier basis:

$$\sin(x), \cos(x)$$

the basis vectors in neural networks adapt during training.
Thus:

Neural networks learn the projection space itself.

Neural Networks as Nonlinear Change of Basis

Classical linear algebra:

$$x \to Wx$$

is a linear change of coordinates.
Neural networks extend this idea to:

$$x \to \sigma(Wx + b)$$

which becomes:

Nonlinear change of basis

Stacking layers repeatedly creates increasingly expressive feature spaces.

Representation Learning View of Architectures

Most architectural innovations in deep learning can be interpreted as attempts to learn better embeddings.

Examples:

- CNNs: learn translation-invariant image representations
- RNNs: learn temporal representations
- Transformers: learn contextual relational embeddings
- Autoencoders: learn compressed latent representations
- Variational Autoencoders: learn probabilistic latent spaces
- Contrastive Learning: learn geometry-preserving embeddings
- Graph Neural Networks: learn relational node embeddings

Geometric Interpretation

Embedding learning can also be viewed geometrically.

Raw data may not be linearly separable in original space:

$$x \in \mathbb{R}^d$$

A learned transformation:

$$z = \phi(x)$$

maps data into a new space where:

- clusters become separable
- semantic structure emerges
- distances become meaningful

Thus representation learning is fundamentally:

Learning a geometry of data

Key Insight

Classical methods:

$$\text{Data} \rightarrow \text{Fixed basis projection}$$

Deep learning:

$$\text{Data} \rightarrow \text{Learned nonlinear transformations}$$

Hence modern deep learning is fundamentally:

Learning representations through adaptive transformations.

Attention Mechanism

In sequential data, a single sample is no longer represented as a single vector.
Instead, it is represented as a collection of tokens.
Examples:

- Words in a sentence
- Patches in an image
- Frames in a video
- Amino acids in a protein

Sequence Representation

Consider a sequence:

$$X = \{x_1, x_2, x_3, \ldots, x_T\}$$

where each token is a d-dimensional vector:

$$x_i \in \mathbb{R}^d$$

Thus a single data sample is represented as a matrix:

$$X \in \mathbb{R}^{T \times d}$$

where:

- T = number of tokens
- d = embedding dimension

Each row of X corresponds to one token embedding.

Objective of Attention

The objective of attention is:

Learn representations that encode interactions between tokens.

Unlike RNNs, attention allows:

- direct token-to-token interaction
- global context modeling
- parallel computation

Attention learns:

$$\phi(X)$$

where every token representation depends on all other tokens.

Learnable Projections

Given input matrix:

$$X \in \mathbb{R}^{T \times d}$$

define three learnable linear projections.

Query Projection

$$Q = XW_Q$$

where

$$W_Q \in \mathbb{R}^{d \times d_q}$$

Thus:

$$Q \in \mathbb{R}^{T \times d_q}$$

Key Projection

$$K = XW_K$$

where

$$W_K \in \mathbb{R}^{d \times d_k}$$

Thus:

$$K \in \mathbb{R}^{T \times d_k}$$

Value Projection

$$V = XW_V$$

where

$$W_V \in \mathbb{R}^{d \times d_v}$$

Thus:

$$V \in \mathbb{R}^{T \times d_v}$$

Common Simplification

Usually we assume:

$$d_q = d_k = d_v = d_{\text{model}}$$

Therefore:

$$Q, K, V \in \mathbb{R}^{T \times d_{\text{model}}}$$

The matrices:

$$W_Q, W_K, W_V$$

are learnable parameters.

Interpretation of Q, K and V

Each row of:

$$Q, K, V$$

corresponds to one token.
For token x_i:

$$q_i, k_i, v_i \in \mathbb{R}^{d_{\text{model}}}$$

Interpretation:

- Query: What information am I looking for?
- Key: What information do I contain?
- Value: What information should I pass forward?

Self Attention

Self-attention computes interactions between tokens within the same sequence.
Each token attends to all other tokens.

Step 1: Similarity Computation

Take one query vector:

$$q \in \mathbb{R}^{d_{\text{model}}}$$

Compute similarity with all key vectors:

$$s_i = qk_i^T$$

for:

$$i = 1, 2, \ldots, T$$

This is a dot product similarity.
Stacking all similarities gives:

$$s = [qk_1^T, qk_2^T, \ldots, qk_T^T]$$

Thus:

$$s \in \mathbb{R}^{1 \times T}$$

Matrix form:

$$s = qK^T$$

Shape computation:

$$(1 \times d_{\text{model}})(d_{\text{model}} \times T) = (1 \times T)$$

Interpretation of Similarity Scores

The score:

$$qk_i^T$$

measures alignment between:

- query token
- key token

Large value:

$$qk_i^T \gg 0$$

means strong similarity.
Small or negative value means weak similarity.
Thus the vector:

$$s$$

quantifies interaction strengths between tokens.

Step 2: Scaling

Normalize the similarity scores:

$$\hat{s} = \frac{s}{\sqrt{d_{\text{model}}}}$$

Why scaling?
Dot products grow with dimension.
If:

$$d_{\text{model}}$$

is large, then:

$$qk_i^T$$

can become very large.
Large values push softmax into saturation, leading to unstable gradients.
Scaling stabilizes training.

Step 3: Softmax Normalization

Convert scores into probabilities:

$$\alpha = \text{softmax}(\hat{s})$$

Component-wise:

$$\alpha_i = \frac{\exp\left(\frac{qk_i^T}{\sqrt{d_{\text{model}}}}\right)}{\sum_{j=1}^{T} \exp\left(\frac{qk_j^T}{\sqrt{d_{\text{model}}}}\right)}$$

Properties:

$$0 \leq \alpha_i \leq 1$$

and

$$\sum_{i=1}^{T} \alpha_i = 1$$

Thus:

$$\alpha$$

represents an attention probability distribution.

Interpretation of Attention Weights

The attention weights determine:

How much importance should be assigned to each token.

Large:

$$\alpha_i$$

means token i is highly relevant to the query token.

Step 4: Attention Weighted Aggregation

Now compute the attention output vector.

$$A_q = \sum_{i=1}^{T} \alpha_i v_i$$

This is a weighted combination of value vectors.
Interpretation:

- Similar tokens contribute more
- Irrelevant tokens contribute less

Thus attention dynamically mixes information across tokens.

Matrix Form of Self Attention

Instead of processing one query at a time, we process all tokens simultaneously.

Similarity Matrix

Compute:

$$QK^T$$

Shape:

$$(T \times d_{\text{model}})(d_{\text{model}} \times T) = (T \times T)$$

Interpretation:

$$(QK^T)_{ij} = q_i k_j^T$$

This matrix contains pairwise similarities between all tokens.

Attention Matrix

Scale and normalize:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{model}}}}\right)$$

where:

$$A \in \mathbb{R}^{T \times T}$$

Each row sums to 1.
Each row defines how one token attends to all others.

Final Attention Output

Multiply by values:

$$Z = AV$$

Thus:

$$Z = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{model}}}}\right)V$$

This is the self-attention equation.

Final Self Attention Equation

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{model}}}}\right)V}$$

Shape Analysis

$$QK^T : (T \times d)(d \times T) = (T \times T)$$

After softmax:

$$A \in \mathbb{R}^{T \times T}$$

Then:

$$AV : (T \times T)(T \times d) = (T \times d)$$

Therefore:

$$Z \in \mathbb{R}^{T \times d}$$

Thus attention preserves sequence structure.

Attention as a Transformation

Attention defines a nonlinear transformation:

$$X \to \phi(X) \to Z$$

where:

$$X \in \mathbb{R}^{T \times d}$$

and

$$Z \in \mathbb{R}^{T \times d}$$

Unlike fixed projections, attention dynamically changes representations depending on token interactions.

Geometric Interpretation

Attention learns:

- relational geometry
- contextual embeddings
- interaction-aware representations

Each token embedding becomes context dependent.
Thus:

$$z_i$$

is no longer just a representation of token x_i.
Instead:

$$z_i$$

contains information aggregated from the entire sequence.

Key Insight

RNNs propagate information sequentially:

$$x_1 \to x_2 \to x_3$$

Attention directly connects all tokens:

$$x_i \leftrightarrow x_j$$

Thus transformers learn global dependencies efficiently.
Attention is fundamentally:

A learnable interaction-based transformation.

Attention as Kernel Density Estimation (KDE)

A very useful interpretation of attention is:

Attention is an adaptive kernel smoother.

More specifically,

$$\boxed{\text{Attention} \approx \text{Kernel Density Estimation (KDE)}}$$

or more generally:

$$\boxed{\text{Attention} = \text{Learned kernel regression in feature space}}$$

Kernel Density Estimation / Kernel Regression

In classical kernel methods, a function is estimated using weighted averages of nearby points.

Given data:

$$\{(x_j, y_j)\}_{j=1}^{N}$$

kernel regression estimates:

$$f(x) = \frac{1}{Z(x)} \sum_{j=1}^{N} K(x, x_j) y_j$$

where:

- $K(x, x_j)$ = similarity kernel
- y_j = value associated with point x_j
- $Z(x)$ = normalization constant

The normalization term is:

$$Z(x) = \sum_{m=1}^{N} K(x, x_m)$$

Thus:

$$f(x) = \sum_{j=1}^{N} \frac{K(x, x_j)}{\sum_m K(x, x_m)} y_j$$

Interpretation of KDE

Kernel regression says:

To estimate a value at point x, take a weighted average of neighboring points.

Nearby points receive larger weights.
Far points receive smaller weights.
The weights are determined by similarity.

Example: Gaussian Kernel

A common kernel is the Gaussian kernel:

$$K(x, x_j) = \exp\left(-\frac{\|x - x_j\|^2}{2\sigma^2}\right)$$

Properties:

- Similar points get high weight
- Distant points get low weight

Thus KDE performs similarity-weighted aggregation.

Attention Equation

Recall self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

For a single query token q_i:

$$A(q_i) = \sum_{j=1}^{T} \alpha_{ij} v_j$$

where

$$\alpha_{ij} = \frac{\exp(q_i k_j^T)}{\sum_{m=1}^{T} \exp(q_i k_m^T)}$$

(ignoring scaling for simplicity).

Matching Attention with KDE

Now compare both equations carefully.

Kernel Regression

$$f(x) = \sum_j \frac{K(x, x_j)}{\sum_m K(x, x_m)} y_j$$

Attention

$$A(q_i) = \sum_j \frac{\exp(q_i k_j^T)}{\sum_m \exp(q_i k_m^T)} v_j$$

The structure is identical.

Term-by-Term Correspondence

Kernel Methods	Attention
Query point x	Query token q_i
Data points x_j	Key tokens k_j
Kernel $K(x, x_j)$	$\exp(q_i k_j^T)$
Values y_j	Value vectors v_j
Normalization $Z(x)$	Softmax denominator
Weighted average	Attention aggregation

Attention Kernel

In attention, the kernel is:

$$K(i, j) = \exp(q_i k_j^T)$$

or with scaling:

$$K(i, j) = \exp\left(\frac{q_i k_j^T}{\sqrt{d}}\right)$$

This is a similarity kernel.
Large dot product:

$$q_i k_j^T$$

means strong similarity.
Thus similar tokens receive larger attention weights.

Why Dot Product Works as Similarity

Recall:

$$q_i k_j^T = \|q_i\| \|k_j\| \cos\theta$$

Thus the dot product measures alignment between vectors.
If vectors point in similar directions:

$$\cos\theta \approx 1$$

then similarity is large.
Thus attention behaves like a learned similarity metric.

Attention as Adaptive KDE

Classical KDE:

- uses fixed kernels
- Gaussian kernels
- Laplacian kernels
- manually designed similarity measures

Attention differs fundamentally.
The similarity function itself is learned.
Queries and keys are:

$$Q = XW_Q$$

$$K = XW_K$$

Thus the kernel becomes:

$$K(i,j) = \exp\left((XW_Q)_i (XW_K)_j^T\right)$$

Hence:

Attention learns the kernel itself.

Learned Feature Space

Classical kernels operate in fixed feature spaces.

Attention first learns:

$$Q, K, V$$

through trainable projections.

Thus attention performs KDE in a learned embedding space.

$$X \to (Q, K, V) \to \text{Kernel interactions}$$

This makes attention highly adaptive.

Geometric Interpretation

Each token asks:

"Which other tokens are most similar to me?"

Then attention computes:

A weighted average of relevant tokens.

Thus every token dynamically gathers contextual information.

Attention as Message Passing

Another interpretation:

Each token sends information to every other token.

Similarity determines:

- how much information flows
- which tokens communicate strongly

Thus attention is also:

Similarity-weighted message passing

This interpretation connects attention with:

- Graph Neural Networks
- Kernel methods
- Nonlocal operators
- Integral transforms

Comparison with Classical KDE

Classical KDE	Attention
Fixed kernel	Learned kernel
Static similarity	Dynamic similarity
Euclidean distance	Learned dot-product geometry
Hand-designed features	Learned representations
Local smoothing	Contextual interaction learning

Key Insight

Attention can be viewed as:

$$\boxed{\text{Adaptive kernel regression over tokens}}$$

Every token says:

"Give me a weighted average of all tokens, where weights depend on similarity to me."

This is precisely the philosophy behind KDE.
The only major difference is:

Attention learns the similarity geometry itself.

Causal Masking

Recall that in autoregressive sequence modeling,
the current token can only depend on previous tokens.
For a sequence:

$$X = \{x_1, x_2, x_3, \ldots, x_T\}$$

the autoregressive factorization is:

$$p(X) = \prod_{t=1}^{T} p(x_t \mid x_{<t})$$

where

$$x_{<t} = \{x_1, x_2, \ldots, x_{t-1}\}$$

Thus:

$$x_t$$

must not access future tokens:

$$x_{t+1}, x_{t+2}, \ldots, x_T$$

during training.

Problem with Standard Self Attention

Standard self-attention computes:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

The similarity matrix:

$$QK^T \in \mathbb{R}^{T\times T}$$

contains interactions between all pairs of tokens.
Thus token x_t can attend to:

- past tokens
- current token
- future tokens

This violates autoregressive causality.

Goal of Causal Masking

We want:

$$x_t$$

to attend only to:

$$x_1, x_2, \ldots, x_t$$

and never to:

$$x_{t+1}, x_{t+2}, \ldots, x_T$$

Thus we need to remove future-token interactions.

Mask Matrix

Define a mask matrix:

$$M \in \mathbb{R}^{T\times T}$$

with elements:

$$M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}$$

Interpretation:

- Allow attention to previous/current tokens
- Block attention to future tokens

Structure of the Causal Mask

For example, if:

$$T = 5$$

then

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

This is called a:

Lower triangular causal mask

Masked Attention

Modify attention scores using element-wise addition:

$$A_M = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + M\right)$$

where:

$$M$$

is added before softmax.

Why the Mask Works

Consider one attention score:

$$s_{ij} = \frac{q_i k_j^T}{\sqrt{d}}$$

After masking:

$$s_{ij}^{\text{masked}} = s_{ij} + M_{ij}$$

If:

$$j > i$$

then:

$$M_{ij} = -\infty$$

Thus:

$$s_{ij}^{\text{masked}} = -\infty$$

Applying softmax:

$$\exp(-\infty) = 0$$

Therefore:

$$\alpha_{ij} = 0$$

Hence future tokens receive zero attention probability.

Masked Softmax

The masked attention weights become:

$$\alpha_{ij} = \frac{\exp\left(\frac{q_i k_j^T}{\sqrt{d}} + M_{ij}\right)}{\sum_{m=1}^{T} \exp\left(\frac{q_i k_m^T}{\sqrt{d}} + M_{im}\right)}$$

If:

$$j > i$$

then:

$$\alpha_{ij} = 0$$

Thus:

$$x_i$$

cannot attend to future tokens.

Example

Suppose we are predicting:

$$x_4$$

Then allowed attention is:

$$x_1, x_2, x_3, x_4$$

Blocked attention:

$$x_5, x_6, \ldots, x_T$$

Thus generation remains causal.

Geometric Interpretation

Standard self-attention:

Every token communicates with every other token.

Causal masking imposes directional flow:

$$x_1 \to x_2 \to x_3 \to \cdots \to x_T$$

Thus transformers become autoregressive sequence models.

Attention Matrix Interpretation

Without masking:

$$A \in \mathbb{R}^{T \times T}$$

is fully dense.
With causal masking:

$$A_M$$

becomes lower triangular.
Future-token interactions are exactly zero.

Final Masked Attention Equation

The final causal self-attention equation is:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + M\right)V}$$

where:

$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

Why Causal Attention is Important

Causal masking enables transformers to perform:

- autoregressive language modeling
- next-token prediction
- sequence generation

Models such as:

- GPT
- LLaMA
- Claude
- Gemini

all use causal masked self-attention.

Key Insight

Attention alone is bidirectional.

Causal masking introduces temporal ordering.

Thus masked attention combines:

- global interaction modeling
- autoregressive causality

This allows transformers to replace RNNs while still preserving sequence generation behavior.

Multi-Head Attention

Single self-attention learns one interaction geometry between tokens.
However, different types of relationships may exist simultaneously.
Examples:

- syntactic relations
- semantic similarity
- long-range dependencies
- positional interactions

A single attention map may not capture all these structures efficiently.
The solution is:

Multi-Head Attention (MHA)

Core Idea

Instead of learning one attention transformation,
learn multiple attention projections in parallel.
Each attention head learns:

- a different embedding space
- a different similarity geometry
- a different interaction structure

Thus different heads attend to different aspects of the sequence.

Input Representation

Let:

$$X \in \mathbb{R}^{T \times d_{\text{model}}}$$

where:

- T = number of tokens
- d_{model} = embedding dimension

Suppose we use:

$$h$$

attention heads.
Define:

$$d_k = d_v = \frac{d_{\text{model}}}{h}$$

Thus each head operates in a lower-dimensional subspace.

Linear Projections for Each Head

For head r:

$$Q_r = XW_r^Q$$

$$K_r = XW_r^K$$

$$V_r = XW_r^V$$

where:

$$W_r^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

$$W_r^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

$$W_r^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

are learnable matrices.

Shapes of Q,K,V

For each head:

$$Q_r, K_r \in \mathbb{R}^{T \times d_k}$$

$$V_r \in \mathbb{R}^{T \times d_v}$$

Thus every head learns a different projection of the same input sequence.

Self Attention for One Head

For head r:

$$\text{head}_r = \text{softmax}\left(\frac{Q_r K_r^T}{\sqrt{d_k}}\right) V_r$$

Shape analysis:

Similarity Matrix

$$Q_r K_r^T : (T \times d_k)(d_k \times T) = (T \times T)$$

This computes pairwise token similarities.

Attention Matrix

$$A_r = \text{softmax}\left(\frac{Q_r K_r^T}{\sqrt{d_k}}\right)$$

where:

$$A_r \in \mathbb{R}^{T \times T}$$

Each row sums to 1.

Attention Output

$$\text{head}_r = A_r V_r$$

Shape:

$$(T \times T)(T \times d_v) = (T \times d_v)$$

Thus:

$$\text{head}_r \in \mathbb{R}^{T \times d_v}$$

Parallel Attention Heads

Now compute all heads simultaneously:

$$\text{head}_1, \text{head}_2, \ldots, \text{head}_h$$

Each head learns different token interactions.
Examples:

- one head may learn grammar
- another head may learn long-range dependencies
- another head may focus on nearby tokens

Concatenation of Heads

Concatenate outputs along feature dimension:

$$\text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)$$

Shape:

$$(T \times d_v) \to (T \times hd_v)$$

Since:

$$hd_v = d_{\text{model}}$$

the concatenated representation becomes:

$$(T \times d_{\text{model}})$$

Final Output Projection

Apply one final learnable projection:

$$Z = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

where:

$$W^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$$

Thus:

$$Z \in \mathbb{R}^{T \times d_{\text{model}}}$$

Final Multi-Head Attention Equation

$$\boxed{\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O}$$

where:

$$\boxed{\text{head}_r = \text{softmax}\left(\frac{Q_r K_r^T}{\sqrt{d_k}}\right) V_r}$$

Compact Form

For all heads:

$$Q_r = XW_r^Q$$

$$K_r = XW_r^K$$

$$V_r = XW_r^V$$

Then:

$$\text{head}_r = \text{Attention}(Q_r, K_r, V_r)$$

Finally:

$$Z = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

Why Multi-Head Attention Works

Single attention:

- learns one similarity geometry

Multi-head attention:

- learns multiple similarity geometries
- learns multiple feature subspaces
- increases representational power

Each head acts like an independent relational embedding learner.

Geometric Interpretation

Each head defines a different projection:

$$X \to (Q_r, K_r, V_r)$$

Thus every head learns:

- its own coordinate system
- its own kernel similarity
- its own contextual embedding geometry

Attention heads therefore act like:

Multiple learned kernel machines operating in parallel.

Connection with Representation Learning

Recall:

$$z = \phi(x)$$

Multi-head attention generalizes this idea.
Each head learns a different transformation:

$$\phi_r(X)$$

Then all representations are merged together.
Thus MHA is:

Parallel relational representation learning.

Causal Multi-Head Attention

For autoregressive transformers, each head uses causal masking.
For head r:

$$\text{head}_r = \text{softmax}\left(\frac{Q_r K_r^T}{\sqrt{d_k}} + M\right) V_r$$

where:

$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

This ensures autoregressive causality.

Complexity of Multi-Head Attention

The dominant operation is:

$$QK^T$$

Complexity:

$$\mathcal{O}(T^2 d)$$

Thus transformers scale quadratically with sequence length.
This motivates:

- sparse attention
- linear attention
- state-space models
- recurrent transformers

Key Insight

Single-head attention learns:

$$\text{one interaction space}$$

Multi-head attention learns:

$$\text{multiple interaction spaces simultaneously}$$

Thus transformers can capture many different relational structures in parallel.

Variants of Multi-Head Attention

Standard Multi-Head Attention (MHA) is powerful, but computationally expensive.

Main bottlenecks:

- large KV-cache during inference
- memory bandwidth
- quadratic attention cost

This motivated several efficient variants:

- Multi-Query Attention (MQA)
- Grouped-Query Attention (GQA)
- Multi-Head Latent Attention (MLA)

All variants attempt to reduce:

$$\text{memory usage} + \text{inference cost}$$

while preserving attention quality.

Standard Multi-Head Attention (Recap)

Input:

$$X \in \mathbb{R}^{T \times d_{\text{model}}}$$

For head r:

$$Q_r = XW_r^Q$$

$$K_r = XW_r^K$$

$$V_r = XW_r^V$$

Attention per head:

$$\text{head}_r = \text{softmax}\left(\frac{Q_r K_r^T}{\sqrt{d_k}}\right) V_r$$

Final output:

$$Z = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O$$

In MHA:

- every head has separate Q, K, V
- KV cache grows with number of heads

KV-cache complexity:

$$\mathcal{O}(hTd_k)$$

This becomes expensive for large h.

Multi-Query Attention (MQA)

Main idea:

> Use multiple query heads, but share one common key-value head.

Thus:

- Queries remain head-specific
- Keys and values are shared

MQA Projections

Queries:

$$Q_r = XW_r^Q$$

for:

$$r = 1, \ldots, h$$

But only one shared key/value:

$$K = XW^K$$

$$V = XW^V$$

Shapes:

$$Q_r \in \mathbb{R}^{T \times d_k}$$

$$K, V \in \mathbb{R}^{T \times d_k}$$

MQA Attention

For head r:

$$\text{head}_r = \text{softmax}\left(\frac{Q_r K^T}{\sqrt{d_k}}\right) V$$

Notice:

- only Q_r changes across heads
- K, V are shared globally

MQA KV Cache Reduction

Standard MHA cache:

$$\mathcal{O}(hTd_k)$$

MQA cache:

$$\mathcal{O}(Td_k)$$

Thus KV memory reduces by factor:

$$h$$

This significantly improves inference speed.

Interpretation of MQA

In MHA:

- every head learns its own similarity geometry

In MQA:

- all heads share one memory space
- queries ask different questions
- but retrieve from same global memory

Grouped Query Attention (GQA)

MQA is memory efficient, but sometimes loses attention diversity.
GQA provides a compromise between:

- MHA
- MQA

Main idea:

Group query heads together and share KV within groups.

GQA Structure

Suppose:

$$h = 16$$

query heads.
Divide into:

$$g = 4$$

groups.
Each group contains:

$$\frac{h}{g} = 4$$

query heads.
Each group shares one KV pair.

GQA Projections

Queries remain head-specific:

$$Q_r = XW_r^Q$$

for:

$$r = 1, \ldots, h$$

But keys/values are shared per group.
For group c:

$$K_c = XW_c^K$$

$$V_c = XW_c^V$$

where:

$$c = 1, \ldots, g$$

GQA Attention

If query head r belongs to group $c(r)$:

$$\text{head}_r = \text{softmax}\left(\frac{Q_r K_{c(r)}^T}{\sqrt{d_k}}\right) V_{c(r)}$$

Thus heads inside the same group share memory.

KV Cache Complexity

MHA:

$$\mathcal{O}(hTd_k)$$

MQA:

$$\mathcal{O}(Td_k)$$

GQA:

$$\mathcal{O}(gTd_k)$$

where:

$$1 \leq g \leq h$$

Thus GQA interpolates between MHA and MQA.

Interpretation of GQA

GQA balances:

- diversity of attention heads
- inference efficiency

Instead of one global memory (MQA),
we now have:

$$g$$

shared memory spaces.

Multi-Head Latent Attention (MLA)

MLA attempts to compress KV representations into a smaller latent space.
Main idea:

Store compressed latent KV states instead of full KV tensors.

This drastically reduces KV-cache memory.

Latent Compression

Suppose original hidden state:

$$X \in \mathbb{R}^{T \times d_{\text{model}}}$$

Instead of directly forming KV:
compress into latent states:

$$C = XW^C$$

where:

$$W^C \in \mathbb{R}^{d_{\text{model}} \times d_c}$$

and:

$$d_c \ll d_{\text{model}}$$

Thus:

$$C \in \mathbb{R}^{T \times d_c}$$

This becomes the latent memory representation.

Latent Key/Value Reconstruction

Now reconstruct keys and values from latent space.

For head r:

$$K_r = CW_r^K$$

$$V_r = CW_r^V$$

where:

$$W_r^K, W_r^V \in \mathbb{R}^{d_c \times d_k}$$

Queries are still computed normally:

$$Q_r = XW_r^Q$$

MLA Attention

Attention becomes:

$$\text{head}_r = \text{softmax}\left(\frac{Q_r K_r^T}{\sqrt{d_k}}\right) V_r$$

but now:

$$K_r, V_r$$

are reconstructed from compressed latent memory.

KV Cache Reduction in MLA

Instead of storing:

$$K, V \in \mathbb{R}^{T \times d_{\text{model}}}$$

store only:

$$C \in \mathbb{R}^{T \times d_c}$$

where:

$$d_c \ll d_{\text{model}}$$

Memory reduction:

$$\mathcal{O}(Td_c)$$

instead of:

$$\mathcal{O}(hTd_k)$$

This is extremely useful for long-context transformers.

Interpretation of MLA

Standard attention:

- stores full token memory

MLA:

- stores compressed latent memory
- reconstructs KV dynamically

Thus MLA behaves like:

Low-rank memory compression for attention.

Geometric View

MHA:

Many independent interaction spaces

MQA:

Many queries, one shared memory space

GQA:

Groups of queries sharing memory

MLA:

Compressed latent memory manifold

Unified Attention Perspective

All these variants modify:

$$Q, K, V$$

construction.
But the core attention equation remains:

$$\text{Attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

The major difference is:

How memory representations are shared or compressed.

Comparison of Methods

Method	KV Sharing	Memory Cost	Diversity
MHA	None	High	High
MQA	Global	Very Low	Lower
GQA	Group-wise	Medium	Medium
MLA	Latent Compression	Very Low	High

Key Insight

Modern attention variants are fundamentally attempts to optimize:

$$\text{representation quality} \quad vs \quad \text{memory/computation cost}$$

while preserving contextual interaction learning.

Training a Transformer Language Model

The objective of autoregressive transformer training is:

Predict the next token given previous tokens.

Given a sequence:

$$X = \{x_1, x_2, x_3, \ldots, x_T\}$$

training is performed using shifted inputs and targets.

Input and Target Sequences

Define:

$$X_{\text{input}} = \{x_1, x_2, x_3, \ldots, x_{T-1}\}$$

and target sequence:

$$Y = \{x_2, x_3, x_4, \ldots, x_T\}$$

Thus the model learns:

$$x_t \to x_{t+1}$$

This is next-token prediction.

Tokenization

Raw text cannot be processed directly.

Text is first converted into tokens.

Example:

```
"Deep learning is powerful"
```

may become:

$$[45, 128, 91, 762]$$

Each integer corresponds to a vocabulary token.

Suppose vocabulary size is:

$$V$$

Then each token index belongs to:

$$\{1, 2, \ldots, V\}$$

Token Embedding

Tokens are discrete integers.

Neural networks require continuous vectors.

Thus each token is mapped into an embedding vector.

Define embedding matrix:

$$E \in \mathbb{R}^{V \times d_{\text{model}}}$$

where:

- V = vocabulary size
- d_{model} = embedding dimension

For token:

$$x_t$$

its embedding is:

$$e_t = E[x_t]$$

Thus:

$$e_t \in \mathbb{R}^{d_{\text{model}}}$$

Sequence Embedding Matrix

Stack embeddings together:

$$X_E = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_{T-1} \end{bmatrix}$$

Thus:

$$X_E \in \mathbb{R}^{(T-1)\times d_{\text{model}}}$$

This becomes the initial transformer representation.

Why Positional Embeddings are Needed

Attention itself is permutation invariant.

Without positional information:

$$(x_1, x_2, x_3)$$

and

$$(x_3, x_1, x_2)$$

would look identical to the transformer.

Thus positional information must be injected explicitly.

Positional Embeddings

Define positional vectors:

$$p_t \in \mathbb{R}^{d_{\text{model}}}$$

for positions:

$$t = 1, 2, \ldots, T-1$$

Stacking them:

$$P \in \mathbb{R}^{(T-1)\times d_{\text{model}}}$$

The input representation becomes:

$$H^{(0)} = X_E + P$$

Thus:

$$H^{(0)} \in \mathbb{R}^{(T-1)\times d_{\text{model}}}$$

Transformer Block

The embedded sequence is passed through transformer blocks.
A transformer block consists of:

- Multi-head self attention
- Residual (skip) connection
- Feed-forward neural network
- Another residual connection
- Layer normalization

Masked Multi-Head Attention

Input:

$$H^{(l)}$$

for layer l.
Compute:

$$Q = H^{(l)}W_Q$$

$$K = H^{(l)}W_K$$

$$V = H^{(l)}W_V$$

Then masked self-attention:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

where:

$$M$$

is the causal mask.

This ensures:

$$x_t$$

can only attend to:

$$x_1, \ldots, x_t$$

Residual / Skip Connections

Attention output is added back to the original input.

$$H_{\text{attn}} = H^{(l)} + A$$

This is called a residual connection.
Interpretation:

$$y = x + f(x)$$

Gradient:

$$\frac{dy}{dx} = 1 + \frac{df}{dx}$$

Thus gradients can flow directly through identity paths.
Residual connections help:

- stabilize training
- prevent vanishing gradients
- train very deep networks

Feed Forward Network (FFN)

After attention, each token independently passes through an MLP.
Typically:

$$\text{FFN}(x) = W_2\sigma(W_1x + b_1) + b_2$$

where:

$$W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{hidden}}}$$

$$W_2 \in \mathbb{R}^{d_{\text{hidden}} \times d_{\text{model}}}$$

Usually:

$$d_{\text{hidden}} \gg d_{\text{model}}$$

This increases representational power.

Second Residual Connection

Again add skip connection:

$$H^{(l+1)} = H_{\text{attn}} + \text{FFN}(H_{\text{attn}})$$

Thus each transformer block learns:

$$H^{(l)} \to H^{(l+1)}$$

Deep Transformer Stack

Stack multiple transformer blocks:

$$H^{(0)} \to H^{(1)} \to H^{(2)} \to \cdots \to H^{(L)}$$

where:

$$L$$

is the number of layers.
The final representation:

$$H^{(L)}$$

contains contextual embeddings for all tokens.

Projection Back to Vocabulary Space

The final hidden state must be converted back into token probabilities.
Define output projection matrix:

$$W_{\text{vocab}} \in \mathbb{R}^{d_{\text{model}} \times V}$$

Compute logits:

$$Z = H^{(L)} W_{\text{vocab}}$$

Thus:

$$Z \in \mathbb{R}^{(T-1) \times V}$$

Each row contains scores over the vocabulary.

Vocabulary Probability Distribution

Convert logits into probabilities:

$$p_t = \text{softmax}(z_t)$$

where:

$$p_t \in \mathbb{R}^V$$

and:

$$\sum_{i=1}^{V} p_{t,i} = 1$$

Interpretation:

$$p_{t,i} = P(x_{t+1} = i \mid x_{\leq t})$$

Thus the model predicts the next token distribution.

Cross Entropy Loss

Suppose target token is:

$$y_t$$

represented as one-hot vector.
The loss at time step t is:

$$\mathcal{L}_t = -\sum_{i=1}^{V} y_{t,i} \log p_{t,i}$$

Since y_t is one-hot:

$$\mathcal{L}_t = -\log p_{t,y_t}$$

Total sequence loss:

$$\boxed{\mathcal{L} = -\sum_{t=1}^{T-1} \log P(x_{t+1} \mid x_{\leq t})}$$

This is exactly maximum likelihood training.

Backpropagation

All parameters are optimized jointly:

- token embeddings
- positional embeddings
- attention matrices
- feed-forward layers
- output projection

Optimization is performed using:

$$\nabla_\theta \mathcal{L}$$

with gradient descent variants such as:

- SGD
- Adam
- AdamW

Training Objective

The transformer learns:

$$\text{Predict the next token from context.}$$

Repeated over massive datasets, the model learns:

- syntax
- semantics
- reasoning patterns
- world structure

through autoregressive next-token prediction alone.

End-to-End Pipeline

The complete transformer training pipeline is:

Tokens

↓

Token Embedding + positional Embedding

↓

Masked Multi-Head Attention

↓

Residual Connections + normalization

↓

FFNN

↓

Residual Connections + normalization

↓

Vocabulary Projection(LM head)

↓

Softmax

↓

Cross Entropy Loss

Key Insight

Transformers are fundamentally:

Massive representation learning systems trained through next-token prediction.

Every layer learns progressively richer contextual embeddings of tokens.

Inference in Transformer Language Models

After training, a transformer learns the conditional probability distribution:

$$P(x_t \mid x_{<t})$$

over the vocabulary.
Inference refers to:

Generating new sequences token by token.

Generation is autoregressive.
At every step:

$$x_t \sim P(x_t \mid x_{<t})$$

The generated token is appended back to the sequence, and the process repeats.

Autoregressive Generation

Suppose initial prompt is:

$$X^{(0)} = \{x_1, x_2, \ldots, x_m\}$$

Transformer outputs logits:

$$z_t \in \mathbb{R}^V$$

where:

$$V$$

is vocabulary size.
Convert logits into probabilities:

$$p_t = \text{softmax}(z_t)$$

Component-wise:

$$p_{t,i} = \frac{\exp(z_{t,i})}{\sum_{j=1}^{V} \exp(z_{t,j})}$$

Thus:

$$p_{t,i} = P(x_{t+1} = i \mid x_{\leq t})$$

Inference strategies determine how the next token is selected from:

$$p_t$$

Greedy Decoding

The simplest strategy is:

Choose the most probable token.

Mathematically:

$$x_{t+1} = \arg\max_i p_{t,i}$$

Equivalently:

$$x_{t+1} = \arg\max_i z_{t,i}$$

since softmax preserves ordering.

Greedy Generation Procedure

At each step:

1. Compute logits
$$z_t$$
2. Compute probabilities
$$p_t = \text{softmax}(z_t)$$
3. Select:
$$x_{t+1} = \arg\max_i p_{t,i}$$
4. Append token to sequence

Advantages of Greedy Decoding

- Fast
- Deterministic
- Simple

Problems with Greedy Decoding

Greedy decoding often produces:

- repetitive text
- low diversity
- locally optimal outputs

Because it always chooses:

$$\max_i p_i$$

without exploration.

Sampling / Multinomial Decoding

Instead of choosing the maximum probability token, sample from the probability distribution.

$$x_{t+1} \sim \text{Categorical}(p_t)$$

Thus:

$$P(x_{t+1} = i) = p_{t,i}$$

This introduces randomness into generation.

Interpretation

High probability tokens are more likely, but lower probability tokens may still be sampled.

Thus sampling increases:

- diversity
- creativity
- variability

Temperature Scaling

Before softmax, logits can be rescaled using temperature.

Define temperature:

$$\tau > 0$$

Modified probabilities:

$$p_{t,i}^{(\tau)} = \frac{\exp(z_{t,i}/\tau)}{\sum_j \exp(z_{t,j}/\tau)}$$

Effect of Temperature

Low Temperature

If:

$$\tau \to 0$$

distribution becomes sharp:

$$p_i^{(\tau)} \to \text{one-hot}$$

Generation becomes nearly deterministic.
Equivalent to greedy decoding.

High Temperature

If:

$$\tau \gg 1$$

distribution becomes flatter.
More randomness is introduced.
Rare tokens become more likely.

Interpretation

Temperature controls:

Exploration vs Exploitation

Low temperature:

safe and deterministic

High temperature:

creative and diverse

Top-k Sampling

Pure sampling may select very low-probability tokens, leading to incoherent text.

Top-k sampling restricts sampling to the:

$$k$$

most probable tokens.

Definition

Let:

$$S_k$$

be the set of top-k highest probability tokens.

Then:

$$p_i^{(k)} = \begin{cases} \frac{p_i}{\sum_{j \in S_k} p_j} & i \in S_k \\ 0 & i \notin S_k \end{cases}$$

Then sample:

$$x_{t+1} \sim p^{(k)}$$

Interpretation

Top-k removes unlikely tokens.

Benefits:

- improves coherence
- prevents nonsensical outputs
- still allows diversity

Nucleus Sampling / Top-p Sampling

Top-k uses a fixed number of candidate tokens.

However probability mass may vary across contexts.

Nucleus sampling instead chooses the smallest token set whose cumulative probability exceeds:

$$p$$

Definition

Sort probabilities:

$$p_{(1)} \geq p_{(2)} \geq \cdots$$

Find smallest set:

$$S_p$$

such that:

$$\sum_{i \in S_p} p_i \geq p$$

Then renormalize:

$$p_i^{(p)} = \begin{cases} \frac{p_i}{\sum_{j \in S_p} p_j} & i \in S_p \\ 0 & i \notin S_p \end{cases}$$

Then sample from:

$$p^{(p)}$$

Interpretation

Top-p dynamically adjusts candidate set size.
If distribution is sharp:

$$|S_p|$$

is small.
If distribution is flat:

$$|S_p|$$

becomes larger.
Thus nucleus sampling adapts to uncertainty.

Comparison of Sampling Strategies

Greedy Decoding

$$x_{t+1} = \arg\max_i p_i$$

- deterministic
- low diversity

Sampling

$$x_{t+1} \sim p$$

- highly diverse
- potentially unstable

Top-k Sampling

$$x_{t+1} \sim p^{(k)}$$

- controlled randomness
- fixed candidate size

Top-p Sampling

$$x_{t+1} \sim p^{(p)}$$

- adaptive candidate size
- balances diversity and coherence

Combined Temperature + Top-p Sampling

Modern LLMs often combine:

- temperature scaling
- top-p sampling

Procedure:

1. Scale logits:
$$z_i \to \frac{z_i}{\tau}$$
2. Compute probabilities:
$$p_i = \text{softmax}(z_i/\tau)$$
3. Restrict to nucleus set:
$$S_p$$
4. Renormalize
5. Sample token

This produces high-quality diverse text.

Sequence Probability During Inference

Generated sequence:

$$X = \{x_1, x_2, \ldots, x_T\}$$

has probability:

$$P(X) = \prod_{t=1}^{T} P(x_t \mid x_{<t})$$

Inference algorithms attempt to approximately maximize or sample from this distribution.

Why Sampling Matters

Training objective:

$$\max_{\theta} \log P(X)$$

learns probability distributions.
Inference determines:

How we traverse that learned distribution.

Different decoding strategies explore different regions of the probability space.

Geometric Interpretation

Transformer outputs a probability manifold over vocabulary space.
Inference strategies define trajectories through this manifold.
Greedy decoding:

highest-probability path

Sampling:

stochastic exploration

Top-k/top-p:

constrained stochastic exploration

Key Insight

Training learns:

$$P(x_t \mid x_{<t})$$

Inference decides:

How to sample from P

Thus decoding strategies are not part of training.
They are algorithms for traversing the learned probability landscape.